JOBSHEET XI
LINKED LIST
https://github.com/onlyyourza/PraktikumASD/tree/main/Jobsheet12

1. Learning Objectives

After completing this practical session, students will be able to:

1. Construct a linked list data structure
2. Implement a linked list in a program
3. Identify problems that can be effectively solved using a linked list

2. Labs Activities

2.1 Experiment 1: Implementing Single Linked List

Allocated times: 30 minutes

In this experiment, we will practice how to create a Single Linked List using Node-based data representation, access the linked list, and implement data insertion methods.

1. In the project created for this semester, create a new package named **week12**.
2. Create the following classes:
   a. `Student00.java`
   b. `Node00.java`
   c. `SingleLinkedList00.java`
   d. `SLLMain00.java`

   Modify 00 with your number
3. Implement **Student** following this class diagram:

Student
nim: String name: String className: String gpa: double
Student() Student (nm: String, name: String, kls: String, ip: double) print(): void

4. The following code is the Student class implementation

```
public class Student {
    String nim, name, className;
    double gpa;

    public Student(){
    }
    public Student(String nm, String nama, String kls, double ip){
```

```
        nim = nm;
        name = nama;
        className = kls;
        gpa = ip;
    }
    void print(){
        System.out.println(nim+" - "+name+" - "+className+" - "+gpa);
    }
}
```

5. The following code snippet depicts the **Node** class implementation. Please make a note that the **Node** class manages the **Student** object data, that is why we create **Student** attribute within **Node** class. **You will need to modify it based on the data type of data that you manage.**

```
public class Node {
   Student data;
   Node next;
   public Node(){
   }
   public Node(Sudent data, Node next){
    this.data = data;
    this.next = next;
   }
}
```

6. Create a new class named **SingleLinkedList**
7. Add **head** attributes within **SingleLinkedList** class

```
public class SingleLinkedList {
    Node head;
    Node tail;

}
```

8. In the next step we will implement the basic method of single linked list
9. Create a new method, **`isEmpty()`**.

```
boolean isEmpty(){
        return (head==null);
    }
```

10. Implement traversal process for displaying all data managed in the single linked list

```
void print(){
        if(!isEmpty()){
            Node tmp = head;
```

```
            System.out.println("LinkedList Data:");
            while(tmp!=null){
                tmp.data.print();
                tmp = tmp.next;
            }
        }else{
            System.out.println("LinkedList is empty!!");
        }
    }
```

11. Add **`addFirst()`** method as follows:

```
void addFirst(Student std){
        Node newNode = new Node(std, null);
        if(isEmpty()){
            head = newNode;
            tail = newNode;
        }else{
            newNode.next = head;
            head = newNode;
        }
    }
```

12. Create a new method, **`addLast()`** as follows:

```
void addLast(Student std){
        Node newNode = new Node(std, null);
        if(isEmpty()){
            head = newNode;
            tail = newNode;
        }else{
            tail.next = newNode;
            tail = newNode;
        }
    }
```

13. Add **`insertAfter()`**, to add a new student data after a **key** specified. In this example, we use student name as the key.

```
void insertAfter(Student std, String key){
        Node newNode = new Node(std, null);
        Node temp = head;
        do {
            if (temp.data.name.equalsIgnoreCase(key)) {
                newNode.next = temp.next;
                temp.next = newNode;
                if (newNode.next == null) {
                    tail = newNode;
                }
```

```
                break;
            }
            temp = temp.next;
        } while (temp != null);
    }
```

14. Add these following codes to add a node based on defined index

```
public void insertAt(int index, Student std) {
        if (index < 0) {
            System.out.println("Wrong index!!");
        } else if (index == 0) {
            addFirst(std);
        } else {
            Node temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = new Node(std, temp.next);
            if (temp.next.next == null) {
                tail = temp.next;
            }
        }
    }
```

15. Create a new **Main** class, with a **`main`** method inside. Then create a **SingleLinkedList** object.

```
public static void main(String[] args) {
        SingleLinkedList sll = new SingleLinkedList();

    }
```

16. Add 4 student objects, **std1**, **std2**, **std3** and **std4**.

```
Student std1 = new Student("001", "Student 1", "TI-1I", 3.89);
Student std2 = new Student("002", "Student 2", "TI-1I", 3.45);
Student std3 = new Student("003", "Student 3", "TI-1I", 3.20);
Student std4 = new Student("004", "Student 4", "TI-1I", 3.00);
```

17. Add the students object into SingleLinkedList object, following these scenarios:

```
sll.print();
sll.addFirst(std4);
sll.print();
sll.addLast(std1);
sll.print();
sll.insertAfter(std3, "Student 4");
sll.insertAt(2, std2);
sll.print();
```

2.1.1 Output Verification

Observe the output of your program and compare it with the following output!

```
LinkedList is empty!!
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
001 - Student 1 - TI-1I - 3.89
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
003 - Student 3 - TI-1I - 3.2
002 - Student 2 - TI-1I - 3.45
001 - Student 1 - TI-1I - 3.89
```

2.1.2 Questions!

1. Why does compiling the program code result in the message "Linked List is Empty" on the first line?
2. Explain the general purpose of the variable temp in each method!
3. Modify the code so that data can be added via keyboard input!
4. What would happen if we did not use the **tail** attribute? Would it affect the code implementation? Please explain.

Answer:

1. Because the sll.print() method is called first, before any data is added to the list, head == null, so isEmpty() returns true, and the print method prints the empty message.

2. The temp variable is used as a traversal pointer to navigate nodes from head to back without changing the head's position. If we directly modify head while traversing, we lose the reference to the beginning of the list and lose the data. So, temp is like a "visitor" walking from carriage to carriage, while head/tail remain as "doormen."

3. modify

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    SingleLinkedList14 sll = new SingleLinkedList14();

    System.out.print("Berapa jumlah mahasiswa yang ingin diinput? ");
    int n = sc.nextInt();
    sc.nextLine();

    for (int i = 0; i < n; i++) {
        System.out.println("\nMahasiswa ke-" + (i + 1));
        System.out.print("NIM        : ");
        String nim = sc.nextLine();
        System.out.print("Nama       : ");
        String nama = sc.nextLine();
        System.out.print("Kelas      : ");
        String kls = sc.nextLine();
        System.out.print("IPK        : ");
        double ip = sc.nextDouble();
        sc.nextLine();

        Student14 std = new Student14(nim, nama, kls, ip);
        sll.addLast(std);
    }
```

4. If the tail is removed, the program can still run functionally, but its performance suffers. For example, in addLast(): without the tail, we have to traverse the entire list from the head until we find the last node before connecting. For a list of 1,000 nodes, we have to loop 1,000 times each time we add a node to the end.

With the tail, we have direct access to the last node, making the addLast and removeLast operations much more efficient. In short: the tail is not mandatory, but an optimization that reduces the complexity from O(n) to O(1) for back-end operations.

2.2 Experiment 2: Accessing Element in Single Linked List

Allocated time: 30 minutes

In this practical session, we will learn how to access elements, retrieve their index, and perform data deletion in a Single Linked List.

2.2.1 Experiment Steps

1. Open **SingleLinkedList** class that is already created in Experiment 1.
2. Add a **getData()** method to get data at a specific index in the SingleLinkedList class.

```
Student getData(int idx){
        if(isEmpty()){
            System.out.println("LinkedList is empty!!");
            return null;
        }
        Node tmp = head;
        for(int i=0; i<idx;i++){
            tmp = tmp.next;
        }
        return tmp.data;
    }
```

3. Add a new method to get the index of a data specified. This method is named **`indexOf()`**.

```
int indexOf(String key){
        if(isEmpty()){
            System.out.println("LinkedList is empty!!");
            return -1;
        }
        Node tmp = head;
        int idx = 0;
        while(tmp != null && !tmp.data.name.equalsIgnoreCase(key)){
            tmp = tmp.next;
            idx++;
        }
        if(tmp == null){
            return -1;
        }else{
            return idx;
        }
    }
```

4. Add a new mwthod named **removeFirst()** to remove the first element in single linked list object

```
void removeFirst(){
        if(isEmpty()){
            System.out.println("LinkedList is empty!!");
        }else if(head==tail){
            head = tail = null;
        }else{
            head = head.next;
        }
    }
```

5. Add a method to remove the last element in the SingleLinkedList class, named **removelast()**

```
void removeLast(){
        if(isEmpty()){
            System.out.println("LinkedList is empty!!");
        }else if(head==tail){
            head = tail = null;
        }else{
            Node tmp = head;
            while(tmp.next != tail){
                tmp = tmp.next;
            }
            tmp.next = null;
            tail = tmp;
        }
```

```
    }
```

6. As the next step, the **remove()** method will be implemented.

```
public void remove(String key) {
        if (isEmpty()) {
            System.out.println("LinkedList is empty!!");
        } else {
            Node temp = head;
            while (temp != null) {
                if ((temp.data.name.equalsIgnoreCase(key)) && (temp ==
head)) {
                    removeFirst();
                    break;
                } else if (temp.next.data.name.equalsIgnoreCase(key)) {
                    temp.next = temp.next.next;
                    if (temp.next == null) {
                        tail = temp;
                    }
                    break;
                }
                temp = temp.next;
            }
        }
    }
```

7. Implement a method to **removeAt()** a node by a specified index.

```
public void removeAt(int index) {
        if (index == 0) {
            removeFirst();
        } else {
            Node temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = temp.next.next;
            if (temp.next == null) {
                tail = temp;
            }
        }
    }
```

8. Next, try accessing and deleting data in the **main** method of the **Main** class by adding the following code.

```
System.out.println("Data at index 1 is:");
Student data = sll.getData(1);
data.print();
```

```
int idx = sll.indexOf("Student 1");
System.out.println("Student 1 is located at index: "+idx);

sll.removeFirst();
sll.removeLast();
sll.print();
sll.removeAt(0);
sll.print();
```

9. Re-run the **Main** class

2.2.2 Output verification

Observe the output of your program and compare it with the following output!

```
Data at index 1 is:
003 - Student 3 - TI-1I - 3.2
Student 1 is located at index: 3
LinkedList Data:
003 - Student 3 - TI-1I - 3.2
002 - Student 2 - TI-1I - 3.45
LinkedList Data:
002 - Student 2 - TI-1I - 3.45
```

2.2.3 Questions

1. Why is the **break** keyword used in the remove function? Explain!
2. Explain the purpose of the code below in the remove method.

```
temp.next = temp.next.next;
if (temp.next == null) {
   tail = temp;
}
```

Answer:

1. Because once the target node is found and deleted, the method's task is complete. Without a break, the loop will continue, which can lead to:

Inefficiency, looping is unnecessary after the data has been found.

Error / NullPointerException because temp.next has changed, accessing temp.next.data in the next iteration can result in an error if it turns out to be null.

Double deletion, if there are duplicate names, all will be deleted even though the user only requested to delete one.

2. The first line (temp.next = temp.next.next) is the essence of node deletion. We "skip" the node we want to delete by connecting temp directly to the node after the target.

The if line checks whether the node being deleted was the last node. If temp.next is null after disconnection, it means temp is now the last node, so tail must be moved to temp. If we forget to update tail, the tail pointer will point to the deleted node.

3. Assignments

Allocated times: 50 menit

Create a queue-based program for student service operations with the following requirements:

a. Implement the queue using a **Linked List-based Queue**.
b. The program should be a **new project**, not a modification of an existing example.
c. When a student wants to join the queue, they must **register their information**.
d. Include functions to **check if the queue is empty**, **check if it is full**, and **clear the queue**.
e. Implement **adding a student to the queue**.
f. Implement **calling the next student in the queue**.
g. Display the **first (front)** and **last (rear)** student in the queue.
h. Display the **total number of students** still in the queue.

ANSWER

```java
package Jobsheet12.Queue;

public class QueueNode14 {
    String nim;
    String name;
    String className;
    double gpa;

    QueueNode14 next; // pointer ke node yang ada di belakang dalam antrian

    public QueueNode14(String nim, String name, String className, double gpa) {
        this.nim       = nim;
        this.name      = name;
        this.className = className;
        this.gpa       = gpa;
        this.next      = null;
    }

    public void print() {
        System.out.println("NIM     : " + nim);
        System.out.println("Nama    : " + name);
        System.out.println("Kelas   : " + className);
        System.out.printf (format: "IPK     : %.2f%n", gpa);
    }
}
```

```java
package Jobsheet12.Queue;

public class LinkedQueue14 {

    private QueueNode14 front;
    private QueueNode14 rear;
    private int size;

    public LinkedQueue14() {
        front = null;
        rear  = null;
        size  = 0;
    }

    // isEmpty – kembalikan true jika antrian kosong
    public boolean isEmpty() {
        return front == null;
    }

    // getSize – jumlah elemen dalam antrian
    public int getSize() {
        return size;
    }

    // enqueue – tambah elemen baru di bagian belakang antrian
    public void enqueue(String nim, String name, String className, double gpa) {
        QueueNode14 newNode = new QueueNode14(nim, name, className, gpa);
        if (isEmpty()) {
            // antrian masih kosong, node baru sekaligus jadi front dan rear
            front = newNode;
            rear  = newNode;
        } else {
            // sambungkan node baru ke belakang, lalu geser rear
            rear.next = newNode;
            rear      = newNode;
        }
        size++;
```

Jobsheet12 > Queue > LinkedQueue14.java > LinkedQueue14 > enqueue(String, String, String, double)

```
3   public class LinkedQueue14 {
26      public void enqueue(String nim, String name, String className, double gpa) {
37          size++;
38          System.out.println("[Enqueue] " + name + " berhasil masuk antrian.");
39      }
40
41      // dequeue – ambil dan hapus elemen paling depan antrian, Kembalikan node yang dihapus, atau null jika kosong
42      public QueueNode14 dequeue() {
43          if (isEmpty()) {
44              System.out.println(x: "Antrian kosong, tidak ada yang bisa di-dequeue.");
45              return null;
46          }
47          QueueNode14 removed = front;
48          front = front.next; // geser front ke node berikutnya
49
50          // jika setelah dihapus antrian jadi kosong, rear juga harus null
51          if (front == null) {
52              rear = null;
53          }
54          size--;
55          System.out.println("[Dequeue] " + removed.name + " keluar dari antrian.");
56          return removed;
57      }
58
59      // peek – lihat elemen paling depan tanpa menghapusnya
60      public QueueNode14 peek() {
61          if (isEmpty()) {
62              System.out.println(x: "Antrian kosong.");
63              return null;
64          }
65          return front;
66      }
67
68      // print – tampilkan seluruh isi antrian dari depan ke belakang
69      public void print() {
70          if (isEmpty()) {
71              System.out.println(x: "Antrian kosong.");
```

```
68      // print – tampilkan seluruh isi antrian dari depan ke belakang
69      public void print() {
70          if (isEmpty()) {
71              System.out.println(x: "Antrian kosong.");
72              return;
73          }
74          System.out.println(x: "Posisi  : DEPAN → BELAKANG");
75          QueueNode14 current = front;
76          int urutan = 1;
77          while (current != null) {
78              System.out.println("[" + urutan + "]");
79              current.print();
80              System.out.println(x: "----------------------------");
81              current = current.next;
82              urutan++;
83          }
84      }
85  }
86
```

Jobsheet12 > Queue > QueueMain14.java > ...

```java
package Jobsheet12.Queue;

import java.util.Scanner;

public class QueueMain14 {
    Run | Debug
    public static void main(String[] args) {
        LinkedQueue14 queue = new LinkedQueue14();
        Scanner sc = new Scanner(System.in);

        // Isi data awal agar langsung bisa dicoba
        queue.enqueue(nim: "2401001", name: "Andi Pratama",  className: "TI-1A", gpa: 3.75);
        queue.enqueue(nim: "2401002", name: "Bella Safira",  className: "TI-1A", gpa: 3.50);
        queue.enqueue(nim: "2401003", name: "Cahyo Nugroho", className: "TI-1B", gpa: 3.20);

        int pilihan;
        do {
            System.out.println(x: "\n======================================");
            System.out.println(x: "   QUEUE (FIFO) BERBASIS LINKED LIST");
            System.out.println("   Jumlah antrian: " + queue.getSize());
            System.out.println(x: "======================================");
            System.out.println(x: "1. Enqueue  – tambah mahasiswa ke antrian");
            System.out.println(x: "2. Dequeue  – layani mahasiswa terdepan");
            System.out.println(x: "3. Peek     – lihat mahasiswa terdepan");
            System.out.println(x: "4. Tampilkan seluruh antrian");
            System.out.println(x: "5. Cek apakah antrian kosong");
            System.out.println(x: "0. Keluar");
            System.out.print(s: "Pilihan: ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {

                case 1: { // enqueue
                    System.out.print(s: "NIM    : "); String nim       = sc.nextLine();
                    System.out.print(s: "Nama   : "); String name      = sc.nextLine();
                    System.out.print(s: "Kelas  : "); String className = sc.nextLine();
```

Jobsheet12 > Queue > QueueMain14.java > ...

```
5   public class QueueMain14 {
6       public static void main(String[] args) {
31              switch (pilihan) {
32
33                  case 1: { // enqueue
34                      System.out.print(s: "NIM    : "); String nim       = sc.nextLine();
35                      System.out.print(s: "Nama   : "); String name      = sc.nextLine();
36                      System.out.print(s: "Kelas  : "); String className = sc.nextLine();
37                      System.out.print(s: "IPK    : "); double gpa       = sc.nextDouble(); sc.nextLine();
38                      queue.enqueue(nim, name, className, gpa);
39                      break;
40                  }
41
42                  case 2: { // dequeue
43                      QueueNode14 keluar = queue.dequeue();
44                      if (keluar != null) {
45                          System.out.println(x: "Data yang keluar dari antrian:");
46                          keluar.print();
47                      }
48                      break;
49                  }
50
51                  case 3: { // peek
52                      QueueNode14 depan = queue.peek();
53                      if (depan != null) {
54                          System.out.println(x: "Mahasiswa di posisi terdepan antrian:");
55                          depan.print();
56                      }
57                      break;
58                  }
59
60                  case 4: { // print
61                      System.out.println(x: "\n--- Isi Antrian Saat Ini ---");
62                      queue.print();
63                      break;
64                  }
65
66                  case 5: { // isEmpty
67                      if (queue.isEmpty()) {
68                          System.out.println(x: "Antrian KOSONG.");
69                      } else {
70                          System.out.println("Antrian TIDAK kosong (berisi " + queue.getSize() + " mahasiswa).");
71                      }
72                      break;
73                  }
74
75                  case 0:
76                      System.out.println(x: "Keluar dari program. Sampai jumpa!");
77                      break;
78
79                  default:
80                      System.out.println(x: "Pilihan tidak valid, coba lagi.");
81              }
82
83          } while (pilihan != 0);
84
85          sc.close();
86      }
87  }
```

```
PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 2\Praktik Algoritma Struktur Data\Jobsheet1-semester2>  & 'C:\Program Files
\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\AppData\Roaming\Code\User\workspaceStorage\
e0ae0740bf9eef90a1485cc4c4061a4f\redhat.java\jdt_ws\Jobsheet1-semester2_bc378dc3\bin' 'Jobsheet12.Queue.QueueMain14'

========================================
   QUEUE (FIFO) BERBASIS LINKED LIST
   Jumlah antrian: 3
========================================
1. Enqueue  ? tambah mahasiswa ke antrian
2. Dequeue  ? layani mahasiswa terdepan
3. Peek     ? lihat mahasiswa terdepan
4. Tampilkan seluruh antrian
5. Cek apakah antrian kosong
0. Keluar
Pilihan:
```